

Management of special services — designing for a changing world

J Chandler

COSMOSS, BT's customer-oriented system for the management of special services, was implemented in early 1994. It is a large 3270-based multiple virtual storage (MVS) system using CICS, DB2 and BT Middleware. Though seen by some as a legacy system cloned from the customer service system (CSS), COSMOSS is very forward looking in its attempts to anticipate and accommodate change. This paper discusses the COSMOSS design approach and in particular how it tackles problems associated with the inevitability of change.

1. Introduction

BT's customer-oriented system for the management of special services (COSMOSS) was developed primarily as a national order-handling and provisioning system targeted at business customers and at the provision of private services in particular. The BT customer-facing division who were funding the development were well aware, however, that tightening commercial constraints would limit the chance of many similar future developments. It was essential that maximum benefit be gained from their investment in COSMOSS. From the outset it was stipulated that:

- the system must be capable of rapid enhancement and refinement,
- it must be capable of handling future provisioning requirements both known and unknown,
- it must be capable of taking on far greater processing loads without serious hardware or software constraints.

The requirement for a single, national database, combined with high anticipated volume of orders to be processed, dictated a mainframe solution. The design team was determined, however, to learn as much as possible from experiences with the customer service system (CSS) [1] and other similar large developments. They needed a system that could perform at the volumes indicated and beyond, would be reliable and sufficiently robust to deliver the quality of service demanded, and yet would deliver the flexibility, scalability and future proofing usually associated with a client/server approach.

In attempting to meet these requirements, the designers decided to borrow concepts from object-orientation theory and apply them pragmatically to the mainframe world [2]. In particular they targeted encapsulation as this seemed to offer significant returns. In so doing they looked at the constraints imposed by Cobol, the standard mainframe programming language. They decided, by means of a precompiler step, to implement language refinements which would enforce a highly modular and strictly encapsulated approach to design and development. This approach would force developers to build software in a different style. Rather than constructing large monolithic chunks of code they were required to make use of dynamically called modules to achieve all data access and manipulation. These modules, referred to as capsules, each provided a number of stateless services to access or manipulate the data. The services, designed, coded and tested independently of the programs they supported, were each capable of being used by many programs.

Refinements to XCall, as the infrastructure software became known, ensured that calls from program to service and from service to service were as flexible as possible. The interfaces were designed to limit the impact and consequently the cost of change. Enhancement to one module would not demand that others using it should even be recompiled unless they wished to take advantage of the change delivered. Other refinements allowed the interface to operate across an LU6.2 link allowing services located on other machines to be called using identical software. This feature ultimately would allow the application to be

distributed across multiple machines and would therefore allow for future scalability or partitioning.

At a higher level, it was decided that wherever possible the applications should be table driven rather than hard coded. The order-handling system, for example, is driven by a set of control tables that define how each product should be handled. They define the attributes to be captured, the validation to be applied and the tasks to be completed before the order can be issued. This approach ensures that new products — even those which vary significantly from the existing portfolio — can be taken on board with minimal, if any, change to the application.

The same approach is applied to the more recently added generic job management sub-system. Here the rules defining the activities to be completed and the dependencies between those activities are defined not within the software but within the data; likewise the rules that determine to which work group a given activity should be assigned. Even the interfaces between COSMOSS and other systems are handled in this way (see Table 1 for a summary of the COSMOSS system features). Rules register the interest of external systems in ‘events’ happening within the system. Consequently changes in the status of activities of a certain type and for certain products may be forwarded to a client system such as the major customer service system (MCSS) [3]. Other changes for a different subset of customers or for different types of activity may be passed to Concert or to BT’s managed network services division, all using the same event-trapping code.

Table 1 The COSMOSS system features.

System:	COSMOSS Customer oriented system for the management of special services
Hardware:	IBM ES9000/9X2 8000 3270-based terminals and PCs 800 PC-based local printers
Software:	MVS/ESA CICS 3.3 DB2 3.1 Cobol II Middleware and XCall
Size:	1.5 M lines of code 400 transactions 1100 programs 80 Gb of data
Throughput:	8000 registered users, 2000 concurrent 1M transactions per day 45 transactions per second at peak 18 000 orders processed per week 200 000 activities processed per week
Performance:	Response time: 0.2 sec (target 0.5 sec) Availability: 99.94% (target 99.80%)

The ultimate success of the approach will depend on the system’s ability to cope with unknown future requirements and to take responsibility for the provisioning requirements of product lines not yet defined. At present, the designers are looking at the challenge of migrating COSMOSS to a three-tier or WWW-based client/server operation. This would introduce a more sophisticated and flexible user interface while maintaining the data and much of the existing processing on the mainframe. It is planned to allow client systems to access and reuse the same encapsulated services as the 3270-based transactions that will operate alongside them.

2. Data encapsulation with Cobol

COSMOSS successfully exploited the business’s existing investment in BT Middleware. The infrastructure software described in Calladine [4] was easily ported to a DB2 environment and enhanced to meet a number of COSMOSS’s special needs. Many of these enhancements have now usefully been used by CSS and other Middleware-based systems. Reuse of Middleware gave the project a significant advantage in terms of development productivity. It also meant that many utilities and standards were already in place at project initiation. This gave the design team time early in the project to consider the question of data access, modularisation and system flexibility more carefully. These issues were not addressed by Middleware as used on CSS and elsewhere.

The first result of these considerations was the development of the XCall infrastructure software. XCall facilitates a highly modular approach to program development in a Cobol environment, allowing some of the design advantages of an object-oriented approach to be exploited during large systems development on a multiple virtual storage (MVS) mainframe.

The COSMOSS application is built in two layers with XCall as the ‘glue’ connecting the layers.

There is a layer of fairly conventional Middleware transactions and batch programs. This contains the majority of business logic and handles all interaction with the user. While these programs would look fairly familiar to a CSS developer, they differ from their CSS equivalents in that they perform no direct database access or update. When they require such access they do so by placing a call to the XCall run-time service and invoking an appropriate service module.

The services available are called methods. They are well-defined stateless services with defined input and output parameters. When placing a call, the client transaction defines appropriate input parameters and states

from where, within its working storage, these are to be populated. It also specifies the attributes it wishes to be returned and where they are to be placed. For some methods it is necessary to specify the number of items to be returned. Simple syntax has been defined for this:

A simple call:

```
XCALL GET-CUST
  INPUT  ( CUST-CUSTOMER-ID FROM GX-CUSTOMER-ID
          CUST-NAME-LENGTH FROM W09-56          )
  OUTPUT ( CUST-NAME-ONE-LINE INTO GX-CUST-NAME  )
END XCALL.
```

A more sophisticated call:

```
XCALL GETM-CBAC-BY-<CUSTID-ADDRID>
  INPUT  ( CBAC-CUSTOMER-ID FROM GX-CUSTOMER-ID
          CBAC-BILLING-ACC-NR FROM GX-NEXT-PAGE
          CBAC-ADDRESS-ID FROM GX-ADDRESS-ID
          )
  OUTPUT ( CUST-BILLING-ACC-NR INTO W01-BILLING-ACC-NR (*)
          CBAC-CUST-OWN-REF INTO W01-CUST-OWN-REF (*)
          CBAC-ACC-PURPOSE INTO W01-ACC-PURPOSE (*)
          CBAC-ACC-STATUS INTO W01-ACC-STATUS (*)
          CBAC-ACC-TYPE INTO W01-ACC-TYPE (*)
          )
  ITEMS ( W04-ROWS-REQ )
END XCALL
```

COSMOSS uses in excess of 2000 methods. They may vary in complexity from simple single-item retrieval services to complex join processes which return many items of data. The update methods may likewise operate on a single database row, may operate on many rows, or may execute more complex business processes — cancelling an order or closing a job, for example.

All methods are totally stateless, however. Their operation is, and must always be, totally ignorant of any

factors other than the well-defined parameters that are passed to them as input and the current state of the database records they access. This strictly enforced rule demands a cultural change for many Cobol programmers. However, as anyone with an understanding of object-oriented programming techniques will well appreciate, this statelessness ensures that the methods are fully encapsulated and consequently is the source of the majority of benefit of the COSMOSS approach over more traditional Cobol programming. It ensures that the logic and rules embedded in the method are totally independent of code located elsewhere. It consequently allows the methods to be reused widely across the system. It also enables changes to be made to the method and the underlying data without impact on other programs. Use of stateless services in this way eliminates the problem caused by the presence of numerous undocumented flags and state indicators inherent in many legacy systems. Approximately 60% of COSMOSS application code is in the transaction layer (see Fig 1).

The second application layer within the COSMOSS architecture contains the methods or services themselves. Methods are grouped into modules based on the data they access or manipulate. These modules are referred to as capsules because they ‘encapsulate’ all processing associated with the particular data item they manage (see Fig 2).

The application developers need not concern themselves with how a particular method is accessed. Each method is coded and tested separately as a Cobol section within the

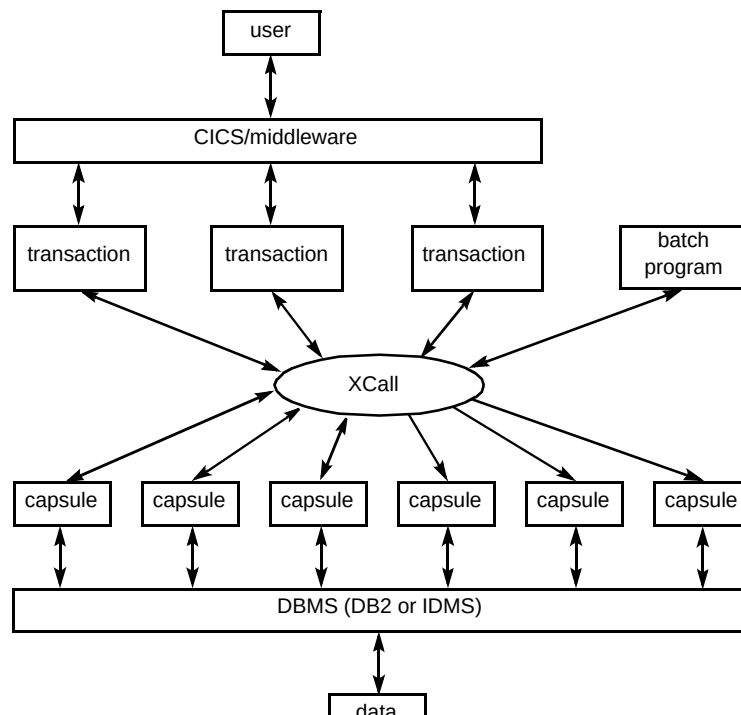


Fig 1 COSMOSS architecture — the application layers.

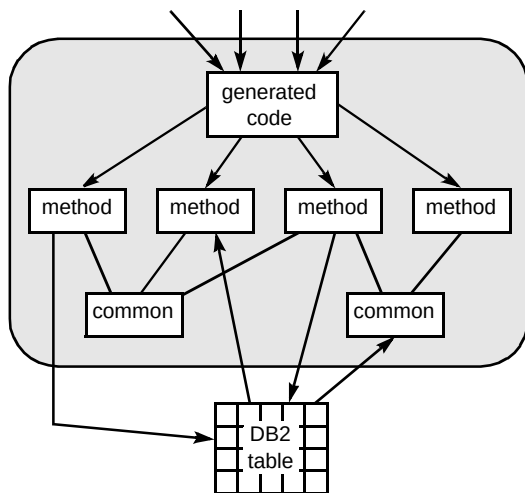


Fig 2 The internal structure of a capsule.

capsule and is identified by having a section name identical to the name of the method itself. Methods may call common code within the capsule and may indeed use XCall to call other methods in the same or other capsules.

Other than coding a few simple declaratives to identify all the methods to the capsule, this is all the programmer supplies. Infrastructure software supplies additional code to pass control around within the capsule and to perform basic validation. A test harness is available to exercise any available method independently of any clients that may use it.

The enlightened reader may well ask what is special about this approach. It is merely the logical extension of modular programming technique and should be standard practice for any good Cobol programmer. One aspect of using XCall that distinguishes it from conventional programming in Cobol is the way it completely decouples called modules from the programs which called them, not only in the sense that the calls are dynamic and consequently may be independently changed, but also that the definition of the data items passed are soft and may themselves also be independently changed. The second feature of note is that the infrastructure automatically enforces the called modules to be completely stateless; it is not dependent on good programming practice. This is partially achieved through the decoupling but is also a result of insisting that all data items passed between the modules are explicitly defined to a central dictionary.

As stated, methods may themselves call upon the services of other methods, and those methods may in turn call others. One rule ensures a degree of sanity and adds significantly to the value of the architecture; all updates and the majority of retrieval processing associated with any item of data must pass through the single capsule which

‘manages’ that data. This rule enforces the encapsulated aspect of the XCall approach. It greatly simplifies the usual problems associated with impact analysis on large mainframe systems and furthermore allows changes to a database table to be implemented while changing only a single application module. No other program need even be recompiled.

This ‘data-hiding’ aspect of the architecture has been taken one stage further. As well as changing the database design (or in theory the database management system) without recompilation of the application code, within reasonable constraints the location of the data can be changed without needing any recompilation. The capsule can be relocated on to another MVS machine accessing a completely separate database. As long as XCall is informed of the move and the necessary communications links are in place, any method request to be serviced by the capsule will be routed via LU6.2 to the appropriate machine. The call will be supported as if it were local to the client application. No change or recompilation of the client application or the capsule is necessary.

The new multi-service billing system is also being developed on an XCall-based architecture. This aspect of software is being used to access the COSMOSS customer database remotely. The billing system has been developed and tested using a local copy of the COSMOSS system. When it is operational, however, requests for COSMOSS data will be picked up by XCall and routed across the network to a remotely located COSMOSS machine.

This facility, coupled with the deliberate partitioning of the COSMOSS database into discrete functional units, provides added future flexibility. Should it ever be necessary, for reasons of system growth or otherwise, it would be possible to split the COSMOSS system across two or more mainframes, each with its own database (see Fig 3). XCall could then be used to route requests for data between the machines as appropriate. COSMOSS has been successfully stress-tested at operational volumes with one database sub-system on another physical machine to prove this concept.

As an element of software infrastructure XCall has three logical components. There is a dictionary where the capsules, the method-based services they provide and the input and output parameters are defined. This is held on a DB2 database and maintained through a series of panels. To avoid excessive database access, the dictionary is not accessed directly by XCall at run time but is used to populate a linear data set which is read into memory at system start-up. The COSMOSS precompiler scans all COSMOSS code prior to it being compiled. With

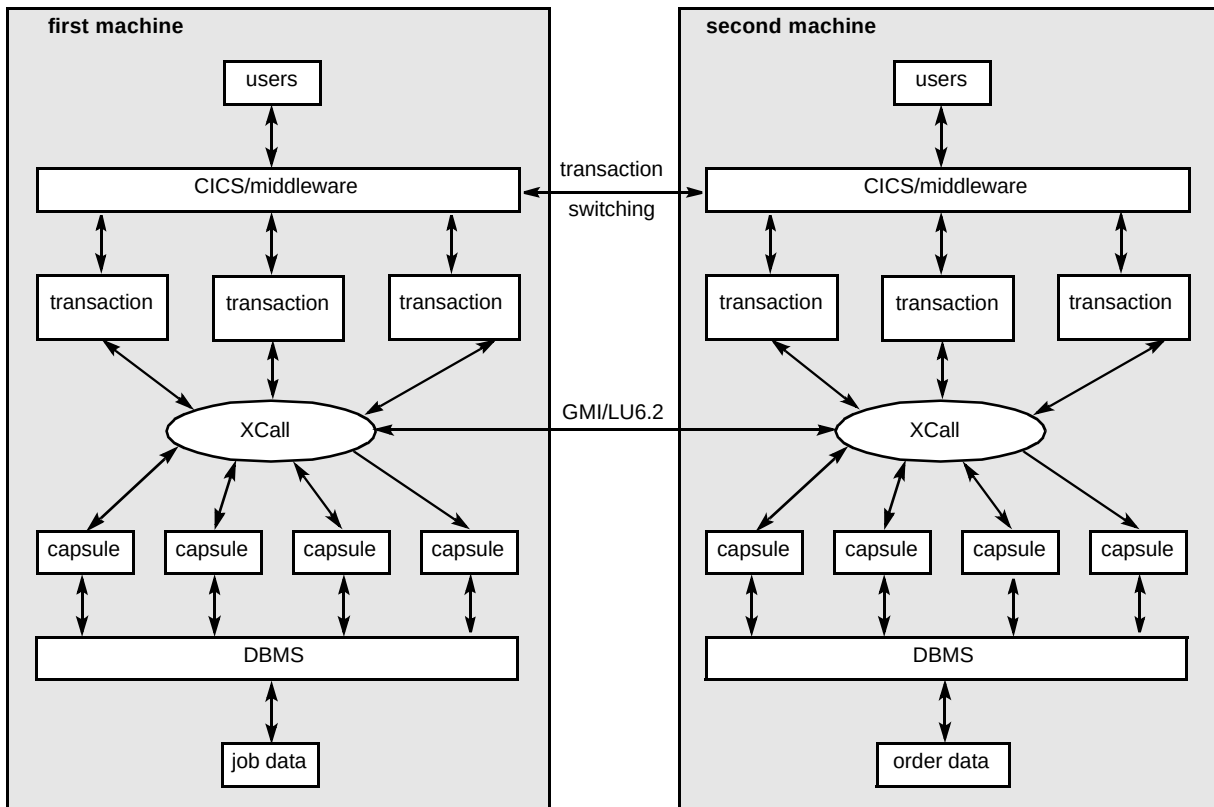


Fig 3 COSMOSS split functionally across two machines.

transactions, batch programs, and capsules that call other capsules, it processes any XCall statements replacing them with the equivalent Cobol statements as necessary. To do this it validates the supplied syntax, checking it against the XCall dictionary, and supplies the appropriate data and procedure division Cobol statements necessary to effect a simple dynamic call to XCall. With capsules, it also has to supply control logic within the module to route method requests to the appropriate application code. The pre-compiler also supplies appropriate error-handling code to trap failures and act as necessary.

In many respects the precompiler is a code generator. Some 30% of the application code within COSMOSS (more

than half a million lines) is precompiler generated rather than hand crafted. The precompiler itself is written in REXX.

Behind the scenes the precompiler also enables a very useful conversion of the individual data items passed from their external public format and names to an internal fixed representation and identification based on the data definition as recorded in the dictionary. This adds another level of protection against change (see Fig 4). It allows different modules within COSMOSS to speak successfully to one another even though they may have been written and compiled at times when the data concerned was defined in a different way. It also ensures that only those items

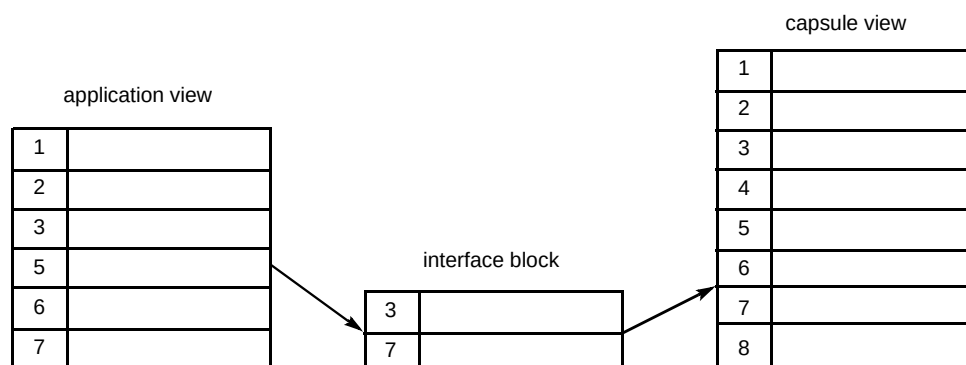


Fig 4 XCall protects against change to data definition.

specifically requested by a client application are passed between the two modules and that only those items of interest to a client are defined in that client. If other items within a record change, the client need not know.

The final component is the XCall run-time module. This is partially written in Assembler. As a typical transaction will invoke XCall a dozen times for each user interaction, the run-time module is called several thousand times a second during COSMOSS operation. The primary function of the run-time system is to route each service request to the appropriate destination capsule and keep control of the flow between modules. This can get very complicated where multiple levels of calls are involved particularly where any call may be to a remote rather than local capsule instance. XCall also plays a large part in the data manipulation necessary to protect client applications from changes to capsules and data discussed above (see Fig 5).

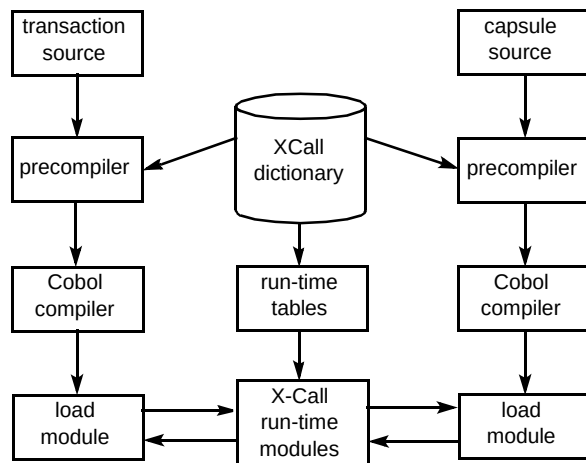


Fig 5 How the XCall components interact.

In the event of calls to remote capsules, XCall uses the Middleware general messaging interface (GMI) to set up an LU6.2 session with XCall on the remote system and manages remote back-out in the event of failure. XCall also enforces constraints on the complexity of remote updates in such situations. Because of limitations in the underpinning software (the lack of a two-phased commit between the two systems), Xcall cannot guarantee that concurrent updates on two separate machines will either both succeed or both fail. The consequences of one succeeding and the other failing may be serious. Incorrect information may be returned or worse, the data may be left in an inconsistent, if not corrupt, state. To avoid such problems occurring, XCall will allow a user to update data on the local machine or on the remote machine but will not allow both to happen simultaneously for the same user transaction.

In summary, XCall allows dynamic calls to be used to their full potential. The design ensures that as far as is possible client modules are protected from changes made to modules they call and data they may access. It allows for

client and server XCall applications to intercommunicate without the caller being aware that the module it is calling is not local. Furthermore, its recovery and abend handling facilities will allow sophisticated remote working and real-time interfaces to be added to COSMOSS with little effort.

These facilities significantly improve the flexibility and maintainability of the COSMOSS application. They limit the impact of change and consequently give developers confidence to proceed with complex coding and design enhancements. They also simplify application logic by reducing complex programs to smaller self-contained modules with clearly defined interfaces. They introduce the possibility of seamless partitioning of the system and hence provide a mechanism for scaling the system to accommodate future growth or re-engineering parts of it without significant impact on the rest.

3. Generic order handling

Within the COSMOSS application the problem of flexibility was tackled in a different but complementary way. During the design the business requirements were subjected to a process of abstraction with the aim of identifying key processes and eliminating as much of the product specific detail as possible. The detail could then be re-accommodated by building the application in a manner that allowed for a sufficient level of customisation.

The best example of this in action can be found in the generic order handling (GOH) sub-system. Though developed primarily to replace the computer automated provision for special services (CAPSS) system, a key requirement was always that COSMOSS should be able to take on board new products and services at minimal cost and in minimal time-scales. In defining this as a priority the COSMOSS client had in mind three factors:

- the onset of competition and the need to get new products to market with ever-reducing lead times,
- the need to reduce costs and the likelihood that money for new support systems would become increasingly hard to obtain,
- the rapid changes in telecommunications technology and the certainty that future products and services may differ significantly from those available today.

To accommodate this requirement, the priority was to handle order-taking in a generic way. COSMOSS is designed such that the functions which can be generalised across all products — customer identification, contact management, etc — are separated from those which require product-specific treatment. For business functions like requirements capture, which cannot be handled in a general way, the system has been built so that all the product-specific processing is determined not by the application

software but by rules held against the product definition on the database. When a customer orders a circuit, for example, the application does not present the user with hard-coded screens to request information such as bit rate, service level, the details of the termination, etc, but instead generalised instructions interrogate the database to determine what questions should be asked for this particular product. Once this has been determined a screen containing appropriate questions is dynamically generated and presented to the user.

The rules to drive this process are held on the database. The product-specific details which are to be captured are recorded against the product type as a table of product features. Each feature has details of how it should be presented at order capture, the type of answer expected, its format, validation and relation to other features.

When an order is taken the product features generate questions and input fields on a dynamically prepared screen and result in product-specific details appropriate to the customer order being captured. These details cannot be held as simple product attributes as they would on a conventional database as they are all product specific rather than general. Instead, they are held separately from the product instance itself as a list of product-specific feature-instance values. Each one is held as a feature type — feature value pair (see Fig 6).

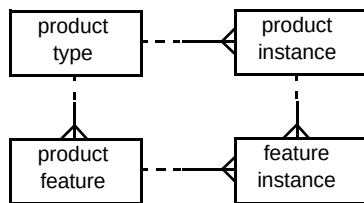


Fig 6 Product data — use of features.

The above description is an oversimplification of the actual design. In reality the rules held against a product are more complex. The grouping of features and the dependencies which can be built between features allow quite complex dialogues to take place at order capture. COSMOSS can consequently be enhanced to accommodate products and services with radically different data composition without any software changes. All the screens required and associated processing can be incorporated by appropriate table changes. While such an approach is not particularly revolutionary it is a radical departure from the designs adopted for most conventional legacy systems.

4. Generic job management (GJM)

With release two of the system the same concepts were extended to the back-office functionality. Job management is concerned with managing and scheduling the tasks associated with the provision of each ordered product and service. With the type of product handled by

COSMOSS the number and variety of tasks to be completed and the allocation of these tasks to engineers can be very complex. It can depend on many factors — geography, type of equipment available, nature of the customer's own equipment, for example. Many of these factors will relate to the product being provisioned and will not be general.

As with generic order handling, in order to provide a flexible solution capable of supporting the provision of future products yet to be specified, the designers adopted a highly rule-driven approach. Jobs arrive in the system in the form of a generic job request. This contains a list of job features with associated values. The template selection process compares the supplied features and values with those held in its selection rules to determine an appropriate template for the job. This template then dictates how the job should be decomposed into individual activities, how much time should be allowed for each activity, and the dependencies between them. Using this template the job request is converted into an actual job and 15—20 activities.

Once the job and activities have been created, each activity has to be assigned to a duty to ensure that it is progressed. This assignment process is again table driven. Feature-based rules dictate criteria for determining how any given activity should be allocated to an organisational unit. For example they might state that 'all widget bashing activities for any green spotted widgets requiring extensions, but excluding any widgets to be located in the London region, must be assigned to the special Manchester-based widget support unit for action'. The rule must obviously be expressed in terms the system can understand, but can be significantly more complex than described. Rules can be expressed using ranges and negatives for example. Automatic selection and allocation can be overridden manually. Sub-templates can be imported into a job from other jobs, etc (see Fig 7).

The result of this table-driven configurability is once again a step further in the direction of flexibility. New job profiles and structures can be defined without application changes. New products can be taken on board and provisioning tasks accommodated even when the management decisions are determined by local rules and new data types.

Within COSMOSS release four a further level of flexibility has been added to job management. New rules may be defined within the template which affect the system behaviour as each activity is closed. As the work defined by an activity is completed the system will generate questions and capture data about the activity concerned. These 'conditions and responses' may simply capture useful data for subsequent display or may impact on the job template itself importing a new set of activities, for example. Again, the questions asked and the system's response are

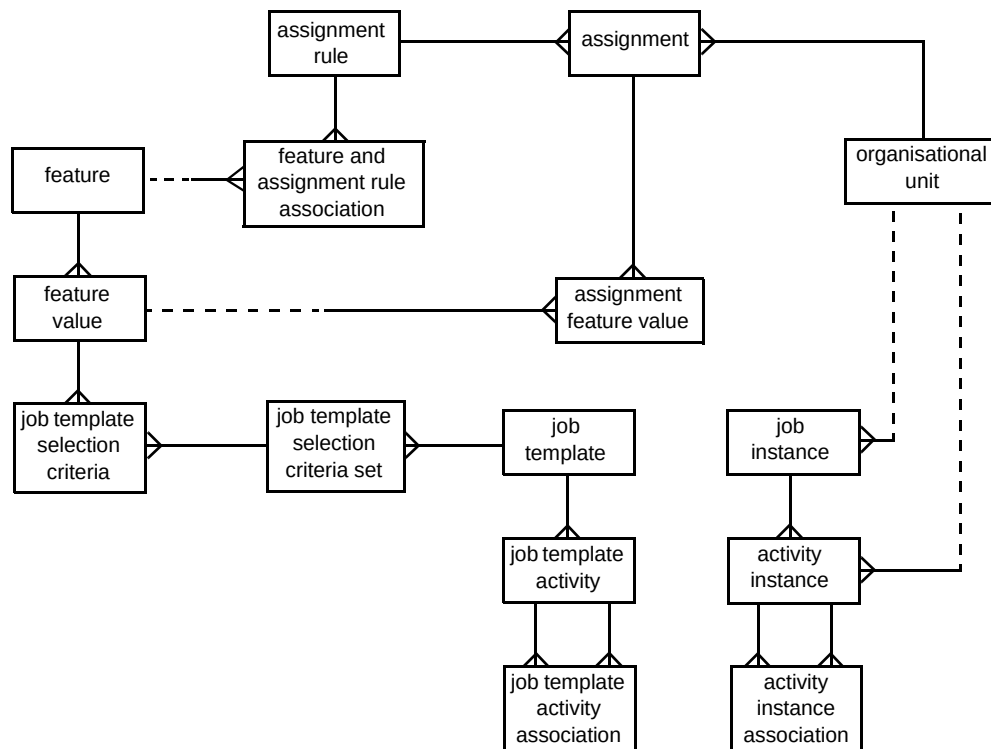


Fig 7 GJM template selection and assignment tables.

configurable and not dictated by the product being provisioned or any other predetermined data.

5. Event-based messaging

As the system has stabilised, an increasing number of other systems require information feeds from COSMOSS. Some of these are major developments such as the COSMOSS/Work Manager interface [5]; others are much smaller initiatives concerning individual product lines. It was realised that most of these interfaces share a common theme:

- they all require real-time asynchronous information feeds from COSMOSS,
- the information is needed whenever an 'event' of some form of interest to the external system happens on COSMOSS.

In the light of previous experience it was obvious that significant gains could be made by tackling such requirements in a generic rather than a specific way. The generic asynchronous interface application (GAIA) attempts to do this. With GAIA operational, the COSMOSS application does not handle the interfaces itself. Instead, when events which may be of interest to other systems occur, they are logged in a generalised way with information such as:

- event type,
- event date and time,
- key information.

Other systems register their interest in logged events by recording rules which can be used to drive event selection in a set of control tables. The data recorded may include:

- the event in which the system is interested,
- conditions determining the events of that type about which they wish to be informed,
- COSMOSS data items (event features) which they wish to be passed for each event,
- the format in which they wish to receive the event information,
- the transport mechanism they wish to be used,
- the times of day and days of the week they are available to receive data.

GAIA then uses these rules to filter the events that are logged, gather additional data and forward it to the systems concerned. Yet again, designing the application in this way means that, as more and more event types are logged, the chances of being able to satisfy new requirements without significant application changes are increased.

6. Has it worked?

The features described here have all been included with the aim of increasing the system's flexibility and reducing the cost of change. Have they succeeded?

From one perspective they are undeniably a success. COSMOSS is operational and is working well. It has very high use, an excellent quality record and impressive performance statistics. The configurable aspects of the system have been well exercised and are effective. There are limitations to the success, however, and they deserve discussion.

The configurability of the system has practical limitations. These have not to date proved an excessive burden, but do inevitably mean that new requirements will frequently demand software changes of one form or another. Some of the constraints could, with some further effort, be removed and will probably be addressed over the coming years. In reality, however, it must be accepted that there is a point beyond which investment of this type has limited return. Possible targets for future improvement are given below.

- Number/resource management — many telephony products require a number to be assigned from a pool of limited availability. Exchange line and top gear (0800) services are obvious examples. Future intelligent network products will almost certainly have equivalent requirements. The problem could be generalised to one of earmarking and allocating a limited resource and could almost certainly benefit from a generalised approach.
- Pricing — the manner in which products and services are priced and discounts applied is highly complex and poorly structured. New products almost invariably demand new pricing algorithms which cannot be supported by existing software. This area would also benefit from review.
- Job requests — the internals of both COSMOSS order handling and COSMOSS job management are both highly generic, but the interface between them and certain aspects of the job management user interface is not. Were GJM to be used to manage jobs from other sources, this might be one area worthy of early attention.
- Project management — while the job creation process within GJM has received a high level of attention other aspects of the design relating to the day-to-day project

management of the jobs are not as configurable. Jeopardy, failure and escalation rules are all hard-coded, for example. Again wider use of the software may result in improvements here.

The assumption has been made that making the system configurable and allowing changes to be made through control table adjustments rather than by software alterations will obviously make the system more flexible and allow business changes to be delivered more rapidly. While this is generally true, one must be cautious not to ignore the radical impact that the resulting configuration changes can have on the systems operation.

There is an equal need to test and control table adjustments as there would be with equivalent software changes. As the system becomes more configurable, the control tables become more complex and the need to validate each change becomes more critical. Table build and table management become significant and onerous responsibilities.

Where software changes are required, the modularity and fragmentation of the system do seem to aid the development process. Without an obvious benchmark, however, it is very difficult to quantify the benefit. There is obvious and effective software reuse with some modules called by fifty or more programs and there are occasional dramatic benefits where changes can be made which would have been prohibitively expensive were a more conventional architecture in place.

On the negative side XCall is a complex and extensive piece of infrastructure software which will continually require dedicated and skilled support. There is minimal possibility of reducing this commitment.

A further concern relates to the management of the software. A high level of modularity inevitably means a large number of interacting software components. These components and the changes to them need to be tracked and controlled. This results in a high configuration-management overhead.

Though COSMOSS has tackled the technical problems associated with rapid software delivery, little has been done to alleviate the management and logistical problems associated with this. Indeed, with increasing demand to reduce delivery times and develop software releases in parallel with one another, the problems are becoming more acute. Unfortunately, while technically COSMOSS may have moved forward, external perceptions are that it is still a large system and is subject to the usual limitations of large releases and long delivery lead times.

7. The way forward

The next major challenge for the COSMOSS designers concerns the migration of computer power to the desktop and the position large mainframe systems such as COSMOSS will play in the architecture of future BT systems. BT is currently committed to a three-tier approach in which large back-end servers acting as custodians of critical business data have a vital role. If COSMOSS is to play an active part, it is essential that it accommodate client/server working by making its data more accessible to client systems.

Fortunately, it appears that the COSMOSS architecture is well suited to client/server working and should aid the system's rapid migration to the role of mainframe database server. It is likely that within a few years the majority of data access requests will come from mid-tier or network-based clients rather than 3270 terminals, as is currently the case.

Two aspects of the architecture should prove beneficial. Early client systems will rely heavily on local databases containing replicated copies of customer data and will access back-end systems only when it is essential to obtain an accurate and up-to-date picture or to update data held there. It will be some time before such systems hold master copies of volatile data and consequently a major concern will be how to ensure that the replicated data is kept in step with its source.

Where COSMOSS is the source system, the GAIA event-based messaging facility is the ideal solution to this problem. Critical data update events can be tracked by GAIA and posted to whichever client systems register an interest. In the case of MCSS for example, GAIA can easily track all new orders placed and all changes in order status or job status. Should such an event affect one of the 500 customers supported by MCSS, details of the change can be despatched to the appropriate MCSS machine within seconds of it happening. These messages can then be used to keep the replicated customer records in step with the master record of that customer held on COSMOSS.

For updates and more sophisticated client system facilities it will be essential to build synchronous links to back-end systems. Existing client applications use tactical mechanisms to achieve this. Many use simple screen scraping tools. Some use the more sophisticated Middleware message-based interface (MMBI) facility [1]. All these mechanisms ultimately rely on driving a screen-based transaction on the mainframe. While these solutions are cheap and effective they have a limited long-term future. There are problems with performance and configuration management, but more critically the client system is forced

to build applications around the constraints imposed by the design of the hub systems' transactions. Where COSMOSS is the database server there is another option, however. COSMOSS transactions themselves use stateless services (methods) to access the database via XCall and these services are already designed to be called remotely if necessary. It is a relatively minor change to allow client systems to access the same services directly. They can then access and manipulate COSMOSS data independently of any transaction (see Fig 8).

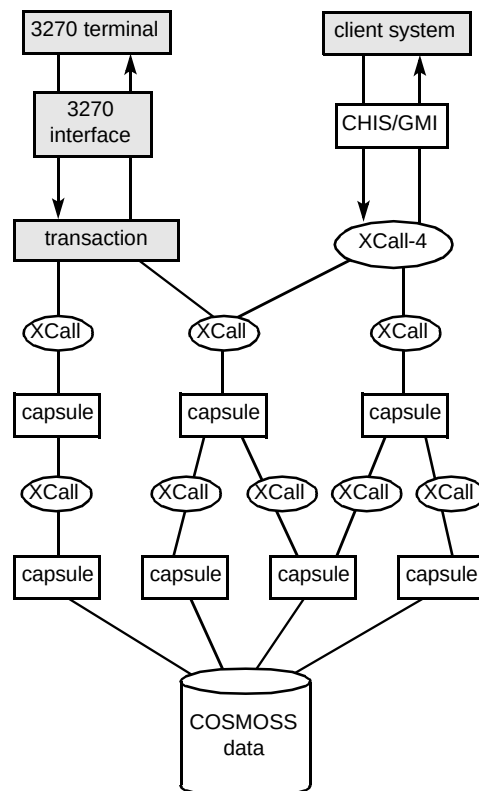


Fig 8 XCall supporting 3270 and client users.

Unfortunately there are a number of problems with this simplistic approach. Existing COSMOSS methods are nearly all fairly low level and perform very basic access or manipulation of individual data items. They enforce minimal validation and very few business rules. As custodian of the data it manages, COSMOSS would not be happy to give client systems access to data at this level. Furthermore, the client system would need to access a number of such methods to perform a single useful business operation. Each call would require a client-to-mainframe remote procedure call with consequent network delays. The response time would be unacceptable. The answer is to build higher-level methods performing business operations by grouping calls to lower-level methods. Such service methods would be of much greater use to client systems. A single remote procedure call could action one such method and perform useful work. Some of these 'service methods'

could themselves be called by re-engineered COSMOSS transactions, others may only ever be of use to remote client systems.

XCall has been used successfully in this way by the new multi-service billing system, but the calls did not pass through the customer handling intermediate server (CHIS) or GMI and consequently the architecture did not fit in well with planned use of the distributed computing environment (DCE) by client systems. The changes to XCall to allow access from mid-tier servers via CHIS and GMI have now been completed, however. The first five such service methods are currently under construction to support MCSS. Further services with more sophisticated functions, such as 'create and issue order', are in the design stage.

It is very early to judge such matters, but early indications suggest the COSMOSS architecture is well placed to accommodate the challenge of client/server operation.

References

- 1 Harrison P F: 'Customer service system — past, present and future', BT Technol J, 15, No 1, pp 29—45 (January 1997).
- 2 Cordingley E S and Dai H: 'Encapsulation — an issue for legacy systems', BT Technol J, 11, No 3, pp 52—64 (July 1993).
- 3 White P: 'A service system for major customers', BT Technol J, 15, No 1, pp (January 1997).
- 4 Calladine J: 'BT Middleware — software as infrastructure', BT Technol J, 15, No 1, pp 81—86 (January 1997).
- 5 Garwood G J: 'Work Manager', BT Technol J, 15, No 1, pp 58—68 (January 1997).



John Chandler was educated at King Charles I Grammar School in Kidderminster and at Durham University. After graduating he stayed at Durham to study for a doctorate in atomic collision theory. This introduced him to mathematical modelling techniques and computing. He joined BT in 1983. Having worked initially with network management applications, he played a key part in the design of the order processing system for Top Gear. He then took responsibility for CSS implementation of the London code change. He joined COSMOSS in the early stages of its specification as technical support manager and has been instrumental in most aspects of its design.